

فاز ۳: مباحث پایه‌ای پیشرفته در جاوا – کامل و

عمیق

این فاز شامل سه ستون اصلی: مدیریت استثنای (Exceptions)، جنریک‌ها (Generics) و کالکشن‌ها (Collections). هر سه توی هر پروژه واقعی روزانه باهاشون سر و کار داری.

3.1 مدیریت استثنای (Exception Handling)

سلسله مراتب استثنای در جاوا

همه چیز از کلاس Throwable ارث می‌بیره که دو تا زیرکلاس اصلی داره:

• **Error**: مشکلات جدی و غیرقابل بازیابی (مثل `OutOfMemoryError`, `StackOverflowError`). برنامه‌نویس نباید و نمی‌تونه کاری براشون بکنه.

• **Exception**: مشکلاتی که برنامه‌نویس می‌تونه مدیریت کنه. خودش دو دسته داره:

• **Checked Exception** (استثنای کنترل‌شده): زیرکلاس مستقیم `Exception` هستند (نه `RuntimeException`). کامپایلر مجبورت می‌کنه یا `try-catch` کنی، یا با `throws` پرتاب کنی.

• **Unchecked Exception** (استثنای کنترل‌نشده): زیرکلاس `RuntimeException` هستند (مثل `NullPointerException`). کامپایلر اجباری برای مدیریتشون نداره.

1 Throwable

2 |— Error (Unchecked)

3 |— Exception

4 |— Checked **Exception** (مثل `IOException`, `SQLException`)

5 |— `RuntimeException` (Unchecked - مثل `NullPointerException`)

! سوال مصاحبه‌ای ۱: Checked Exception و Unchecked Exception رو با مثال توضیح بده. فلسفه وجودیشون چیه؟

جواب:

• Checked: برای شرایطی که قابل پیش‌بینی و خارج از کنترل برنامه هستن، ولی برنامه می‌تونه از پشنون بریاد. (مثلاً فایل پیدا نشد: FileNotFoundException، یا قطع شدن شبکه). طراح جاوا می‌خواسته برنامه‌نویس رو مجبور کنه به این فکر کنه که اگه فایل نبود چی بشه.

• Unchecked: معمولاً ناشی از باگ‌های برنامه‌نویسی هست (مثلاً دسترسی به ایندکس اشتباه، فراخوانی متد روی null). انتظار میره برنامه‌نویس کد رو درست بنویسه که این اتفاق نیفته، نه اینکه همش try-catch بذاره.

```
1 // Checked Exception: مجبوری مدیریتش کنی
2 try {
3     FileReader file = new FileReader("file.txt"); // اگه فایل نباشه چی؟
4 } catch (FileNotFoundException e) {
5     // بازیابی از خطا
6 }
7
8 // Unchecked Exception: اجباری نیست
9 String s = null;
10 s.length(); // خودت مقصری - NullPointerException پرتاب
```

! سوال مصاحبه‌ای ۲: finally چه زمانی اجرا می‌شه؟ تنها زمانی که اجرا نمی‌شه کی هست؟

جواب: بلوک finally همیشه بعد از بلوک try و catch اجرا می‌شه، چه استثنا رخ بده، چه نده، چه catch بشه، چه نشه. حتی اگه توی try دستور return داشته باشیم، finally درست قبل از خروج از متد اجرا میشه.

تنها موردی که اجرا نمی‌شود: اگر `System.exit()` توی `try` یا `catch` صدا بشه، JVM کلاً خاموش میشه و `finally` اجرا نمی‌شه. یا اگر نخ اعدام بشه. کاربرد اصلیش بستن منابع (فایل، کانکشن دیتابیس) هست.

```
1 try {
2     System.out.println("Try block");
3     return; // اجرا میشه finally هم return حتی با
4 } finally {
5     System.out.println("Finally block"); // این پرینت میشه
6 }
```

! سوال مصاحبه‌ای ۳: Try-with-Resources چیه و چه مشکلی رو حل می‌کنه؟

جواب: از Java 7 اضافه شد. مشکل اصلی `finally` این بود که برای بستن هر منبع، کلی کد تکراری و زشت می‌نوشتیم و خود بستن هم می‌تونست استثنا بده (Suppressed Exceptions). Try-with-Resources اینو اتوماتیک می‌کنه. هر کلاسی که `AutoCloseable` رو پیاده‌سازی کنه، می‌تونه توی پرانتز `try` قرار بگیره و `close()` ش به طور خودکار و امن صدا زده میشه.

```
1 // روش قدیمی و زشت a.
2 FileInputStream fis = null;
3 try {
4     fis = new FileInputStream("file.txt");
5 } catch (IOException e) { ... }
6 finally {
7     if (fis != null) {
8         try { fis.close(); } catch (IOException e) { ... }
9     }
10 }
```

```

1
2 // روش جدید و ترجیح داده شده (Try-with-Resources)
3 try (FileInputStream fis = new FileInputStream("file.txt")) {
4     // استفاده از fis
5 } catch (IOException e) {
6     // کار بسته می‌شود
7 }

```

3.2 جنریک‌ها (Generics)

هدف اصلی: فراهم کردن Type Safety (امنیت نوع) در زمان کامپایل، و حذف نیاز به Cast کردن دستی.

تصور کن قبل از Java 5 چه فاجعه‌ای بود:

```

1 // a. قبل از جنریک (Java 1.4)
2 List list = new ArrayList();
3 list.add("Hello");
4 list.add(123); // کامپایلر هیچ اعتراضی ندارد!
5
6 String s = (String) list.get(0); // نیاز به Cast
7 String s2 = (String) list.get(1); // خطا در زمان اجرا! ClassCastException

```

اما با جنریک

```

1 List<String> list = new ArrayList<>();
2 list.add("Hello");
3 // list.add(123); // خطای کامپایل! عالیه

```

1 `String s = list.get(0);` // نیست *Cast* نیازی به

! سوال مصاحبه‌ای ۴: Type Erasure (پاک‌سازی نوع) چیه؟

جواب: جنریک‌ها یه ویژگی زمان کامپایل هستن! برای حفظ سازگاری با نسخه‌های قدیمی جاوا، کامپایلر:

1. نوع‌های جنریک رو چک می‌کنه.

2. همه اطلاعات نوع رو پاک می‌کنه.

3. هر جا لازم باشه، Cast اضافه می‌کنه.

پس در زمان اجرا (Runtime)، JVM اصلاً خبر نداره که `List<String>` بوده یا `List<Integer>`. هر دو فقط `List` هستن. برای همین نمی‌تونی `instanceof` روی نوع جنریک انجام بدی (`(list instanceof List<String>)` if غلطه).

! سوال مصاحبه‌ای ۵: Wildcard ها (?) در جنریک چیا هستن؟ ? extends و ? super رو

توضیح بده (PECS)

جواب: وقتی یه متد می‌نویسی، نمی‌خواهی محدود به یه نوع خاص باشی، اما نمی‌خواهی Type Safety رو هم از دست بدی. اینجا Wildcard ها میان:

• `? extends T` (Upper-Bounded Wildcard): یعنی "هر نوعی که زیرکلاس T باشه (یا خود T)". از این ساختار

فقط می‌تونی بخونی (`get`). نمی‌تونی توی لیست چیزی بذاری (`add`)، چون نمیدونی دقیقاً چه نوعیه.

• `? super T` (Lower-Bounded Wildcard): یعنی "هر نوعی که اَبَرکلاس T باشه (یا خود T)". توی این ساختار

می‌تونی بذاری (`add` از نوع T).

PECS: Producer Extends, Consumer Super

• اگه می‌خواهی از یه کالکشن بخونی (Producer)، از `extends` استفاده کن.

• اگه می‌خواهی توی یه کالکشن بنویسی (Consumer)، از `super` استفاده کن.

```

1 // Producer Extends: فقط میتونم اعداد رو بخونم
2 public void printNumbers(List<? extends Number> list) {
3     for (Number n : list) { // فقط خواندن
4         System.out.println(n);
5     }
6     // list.add(10); // خطا!
7 }
8
9 // Consumer Super: اضافه کنم میتونم بهش Integer
10 public void addNumbers(List<? super Integer> list) {
11     list.add(10); // نوشتن Integer
12     // Integer x = list.get(0); // نوع بازگشتی! خطا!
13 }

```

3.3 کالکشن‌ها (Collections Framework)

این چارچوب یه معماری یکپارچه برای کار با گروهی از اشیاء است. نقشه راه:

• Collection (Interface): ریشه. زیرمجموعه‌هاش: List, Set, Queue.

• Map (Interface): مستقل از Collection. برای زوج‌های کلید-مقدار.

3.3.1 لیست‌ها (Lists)

ArrayList در مقابل LinkedList

! سوال مصاحبه‌ای ۶: ArrayList و LinkedList رو کامل مقایسه کن. کی کدوم رو انتخاب می‌کنی؟

جواب:

ArrayList: در داخل به آرایه پویا هست. المان‌ها پشت سر هم توی حافظه‌ان.

• $O(1)$: $get(index)$ — دسترسی تصادفی فوق‌العاده سریع.

• $O(1)$ amortized: $add(element)$ — اضافه کردن به آخر سریع، ولی اگر ظرفیت آرایه پر بشه، به آرایه بزرگتر ساخته و همه چیز کپی میشه.

• $O(n)$: $add(index, element)$ — اضافه کردن به وسط کند هست چون بقیه المان‌ها باید جابجا بشن.

LinkedList: در داخل به لیست پیوندی دوطرفه هست. هر گره به قبلی و بعدی اشاره داره.

• $O(n)$: $get(index)$ — باید از اول لیست تا اون ایندکس حرکت کنه.

• $O(1)$: $add(index, element)$ — به شرطی که قبلاً به اون گره رسیده باشی. خود عمل درج/حذف سریع.

• $O(1)$: $addFirst/removeFirst$. برای پیاده‌سازی Stack و Queue خیلی خوبه.

چه موقع کدوم؟

• عملیات غالب get و set هست: حتماً ArrayList.

• عملیات غالب اضافه/حذف از وسط یا ابتدای لیست خیلی بزرگه: LinkedList.

• توی عمل و دنیای واقعی، ArrayList به خاطر حافظه کش CPU معمولاً انتخاب بهتری هست و LinkedList کمتر استفاده میشه.

3.3.2 مجموعه‌ها (Sets)

TreeSet و HashSet، LinkedHashMap

• HashSet: از HashMap در داخل استفاده می‌کند. ترتیب رو تضمین نمی‌کند.

عملیات add, remove, contains با $O(1)$.

• LinkedHashMap: مثل HashSet، ولی به لیست پیوندی هم نگه می‌دارد که ترتیب درج (Insertion Order) رو حفظ کند.

• TreeSet: از به درخت سرخ-سیاه (Red-Black Tree) استفاده می‌کند. ترتیب طبیعی (Natural Ordering) یا ترتیب به Comparator که خودت میدی. عملیات اصلی با $O(\log n)$. برای وقتی خوبه که نیاز به چیزای مرتب داری.

3.3.3 نقشه‌ها (Maps)

Hashtable و HashMap، LinkedHashMap، TreeMap

! سوال مصاحبه‌ای ۷: HashMap دقیقاً چطور کار می‌کند؟ (سوال امتیازآور)

جواب:

1. HashMap به آرایه از سطل‌ها (Buckets) هست. هر سطل به Node (کلید، مقدار، اشاره‌گر به بعدی).

2. وقتی put(key, value) می‌کنیم:

• متد hashCode() روی کلید صدا زده میشه.

• به تابع هش ثانویه، این عدد رو به به ایندکس در آرایه تبدیل می‌کند.

• اگه اون سطل خالی بود، گره جدید اونجا ذخیره میشه.

•اگه پر بود (Collision)، به انتهای لیست پیوندی در اون سطل اضافه میشه. در Java 8، اگه تعداد گره‌های یه سطل از ۸ بیشتر بشه، اون لیست پیوندی به یه درخت سرخ-سیاه (Red-Black Tree) تبدیل میشه تا جستجو از $O(n)$ به $O(\log n)$ بهبود پیدا کنه.

3. وقتی `get(key)` می‌کنیم:

•دوباره `hashCode()` صدا زده میشه و ایندکس سطل پیدا میشه.

•اول با `hashCode` مقایسه می‌کنه، سپس برای اطمینان کامل، با `equals()` کلید داخل اون سطل رو با کلید ورودی مقایسه می‌کنه.

! سوال مصاحبه‌ای ۸: چرا باید `hashCode()` و `equals()` رو همزمان با هم Override کنیم؟ (سوال حیاتی)

جواب: بخاطر قرارداد (Contract) بین این دو متد:

•اگه `equals()` میگه دو شیء برابر هستن، `hashCode()` آن‌ها باید یک عدد یکسان برگردونه.

•اگه `hashCode()` دو شیء یکسان باشه، `equals` ممکنه برابر باشه یا نباشه.

عاقبت نقض قرارداد: اگه کلیدی رو با `put` ذخیره کنی، و بعداً همون کلید منطقی (با `equals` برابر) رو با `get` جستجو کنی، ولی `hashCode` ش فرق کنه، `HashMap` توی سطل اشتباه می‌گرده و چیزی پیدا نمی‌کنه. انگار کلیدت گم شده!

! سوال مصاحبه‌ای ۹: `Hashtable` و `ConcurrentHashMap` چه فرقی دارن؟ کدوم بهتره؟

جواب:

•`Hashtable`: قدیمی و از Java 1.0 هست. تمام متدهاش با `synchronized` هستن. یعنی در هر لحظه فقط یک نخ می‌تونه به کل `Map` دسترسی داشته باشه. `Performance` خیلی پایینی داره. کلید و مقدار `null` رو قبول نمی‌کنه.

•ConcurrentHashMap: مدرن و از Java 5. برای محیط‌های هم‌رند طراحی شده. به جای قفل کردن کل ساختار، فقط سطل مورد نظر (bucket-level locking) رو قفل می‌کنه. این کار باعث میشه چندین نخ بتونن همزمان روی سطل‌های مختلف کار کنن. Performance بسیار بالاتری داره. null رو هم قبول نمی‌کنه.

! سوال مصاحبه‌ای ۱۰: متدهای equals و hashCode برای یه کلاس سفارشی چگونه

درست بنویسیم؟

```
1 public class Student {
2     private int id;
3     private String name;
4
5     // برای مقایسه منطقی equals
6     @Override
7     public boolean equals(Object o) {
8         if (this == o) return true; // بهینه‌سازی
9         if (o == null || getClass() != o.getClass()) return false; // نوع چک
10        Student student = (Student) o; // Cast
11        return id == student.id; // مقایسه بر اساس کلید اصلی (id)
12    }
13
14    // hashCode منطبق با equals
15    @Override
16    public int hashCode() {
17        return Objects.hash(id); // فقط از id استفاده کن
18    }
19 }
```